ROOTCAUSE SERVER

18 · Guía para un nuevo desarrollador

Documento: 18-new-developer-guide.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

- Día 1 · Entender de qué va esto
- Día 2 · Entorno funcionando
  - Preparación
  - Comprobar que todo pasa antes de tocar nada
  - Levantar el sistema completo
- Día 3 · Cómo está organizado
- Día 4 · Seguir un flujo completo
- Dónde añadir cosas
  - Una regla de detección nueva
  - Un endpoint nuevo
  - Un dato nuevo en la telemetría
  - Una vista de la consola
- Cómo escribir pruebas aquí
- Partes que requieren especial cuidado
- Convenciones que hay que respetar
- Antes de abrir un PR
- Primeras tareas apropiadas
  - Para empezar el mismo día
  - Primera semana
  - Primer mes
  - Lo que no conviene como primera tarea
- Itinerario resumido
- Dónde preguntar

> Itinerario de incorporación al proyecto: qué leer, en qué orden, cómo dejar el entorno funcionando, cómo seguir un flujo completo, dónde añadir cosas, qué convenciones respetar y qué tocar con especial cuidado.

Día 1 · Entender de qué va esto

Lee en este orden. Está pensado para que cada documento tenga sentido con lo del anterior.

#	Qué leer	Por qué
1	`README.md`	El proyecto en su propia voz, con el ejemplo que lo explica todo
2	01 · Descripción general	Panorama completo, incluida la sección para no técnicos
3	`AGENTS.md`	**Lectura obligatoria.** Los límites que no se negocian

#	Qué leer	Por qué
4	`docs/CAPABILITIES.md`	Qué existe y qué no. Evita proponer cosas ya descartadas
5	03 · Arquitectura	Capas, diagramas y patrones
6	`docs/DETECCION_AMENAZAS.md`	El catálogo real, amenaza por amenaza

Reserva medio día para esto. No es papeleo: `AGENTS.md` y `CAPABILITIES.md` te van a ahorrar proponer tres cosas que el proyecto rechaza por diseño.

Las tres ideas que hay que interiorizar:

1. **`rootcause-core` no toca el mundo.** Ni red, ni archivos, ni reloj propio. Si tu cambio necesita E/S dentro de `core`, el diseño está mal.
2. **Nunca reportar cero donde no se pudo mirar.** Es REQ-SEC-003 y atraviesa todo el producto.
3. **El runbook se escribe, no se ejecuta.** Ninguna acción destructiva automática, nunca.

Día 2 · Entorno funcionando

Preparación

```
git clone https://github.com/vladimiracunadev-create/rootcause-server.git
```

```
cd rootcause-server && cargo build --workspace --locked
```

`rust-toolchain.toml` instala Rust `1.95.0` por ti. Si el enlazador falla en Windows, instala las *Build Tools* de Visual Studio con la carga de C++, o compila dentro de WSL.

Comprobar que todo pasa antes de tocar nada

```
cargo test --workspace --all-targets --locked
```

Debe decir **253 pruebas, 0 fallos**. Si no es así, el problema es del entorno, no tuyo: arréglalo antes de seguir.

Levantar el sistema completo

Terminal 1:

```
bash scripts/dev.sh
```

Copia el token que imprime. Terminal 2:

```
export ROOTCAUSE_API_TOKEN="pega-aqui-el-token"
```

```
cargo run -p rootcause-agent -- --once --label role=internal
```

Abre `http://127.0.0.1:8080`, pega el token, y ya tienes tu propia máquina enrolada como activo.

Prueba esto antes que nada:

```
cargo run -p rootcause-agent -- --dry-run
```

Imprime el sobre exacto que se enviaría. Es la forma más rápida de entender qué datos maneja el sistema.

Y esto:

```
cargo run -p rootcause-server -- rules
```

Las 18 reglas con la pregunta que responde cada una.

Día 3 · Cómo está organizado

```
crates/
  rootcause-core/     dominio y detección · SIN E/S
    models.rs         los contratos que todos comparten
    security.rs       lo que el agente observó
    policy.rs         los umbrales, validados
    detect/           las 18 reglas, una familia por archivo
    posture.rs        la puntuación y su letra pequeña
    runbook.rs        la respuesta guiada
  rootcause-agent/    el sensor, uno por servidor
    probe.rs          EL ÚNICO lugar que ejecuta un comando externo
    net.rs            sockets y pares, un parser por plataforma
    authlog.rs        autenticación, agregada antes de salir del host
    integrity.rs      huellas SHA-256
    baseline.rs       firewall y parches
    collector.rs      un ciclo completo
  rootcause-server/   el plano de control
    config.rs         contrato de arranque; rechaza lo inseguro
    defense.rs        perímetro propio
    auth.rs           tasa → bloqueo → token, en ese orden
    api.rs            19 rutas
    storage.rs        todo el SQL
    watchdog.rs       silencio, poda y retención
console/              la consola, compilada dentro del binario
scripts/              guardianes, humo, SBOM
```

Regla mental para orientarte: si es una decisión (¿esto es grave? ¿esto es público?), está en `core`. Si es una observación (¿qué hay abierto?), está en `agent`. Si es transporte, almacenamiento o presentación, está en `server`.

Día 4 · Seguir un flujo completo

Recorre el camino de un dato de punta a punta. Es la mejor hora invertida de la semana.

Elige el ejemplo del PostgreSQL expuesto y sigue estas paradas:

#	Archivo	Qué mirar
1	`agent/net.rs` → `parse_ss`	Cómo una línea de `ss` se convierte en `ListeningSocket`
2	`core/security.rs` → `BindScope::classify`	Cómo `0.0.0.0` se convierte en `Public`
3	`agent/collector.rs` → `security()`	Cómo se junta todo, incluidas las brechas
4	`agent/main.rs` → `send_once`	Cómo sale del equipo
5	`server/auth.rs` → `require_auth`	Tasa, bloqueo, token
6	`server/api.rs` → `ingest_telemetry`	Las seis validaciones y el ciclo completo
7	`core/policy.rs` → `exposure_severity`	Por qué es crítico y no medio
8	`core/detect/exposure.rs` → `detect`	Cómo nace el candidato
9	`core/runbook.rs` → `restrict_listener`	De dónde salen los comandos
10	`server/storage.rs` → `upsert_incident`	Cómo se deduplica
11	`console/app.js` → `renderExposure`	Cómo llega a la pantalla

La explicación narrada de ese mismo recorrido está en 08 · Flujo de datos.

Después, lee las pruebas de integración. `tests/control_plane.rs` es la mejor especificación ejecutable del producto: 25 pruebas con nombres que describen comportamiento.

Dónde añadir cosas

Una regla de detección nueva

1. Añade su `RuleInfo` a `RULES` en `detect/mod.rs`: id **con el prefijo de su categoría**, título, la pregunta operativa que responde, el techo de severidad y sus técnicas MITRE ATT&CK.
2. Implementa la detección en el archivo de su familia (`exposure.rs`, `intrusion.rs`, `integrity.rs`, `hygiene.rs`, `resource.rs`).
3. Si necesita umbrales, añádelos a `policy.rs` **con validación** en `DetectionPolicy::validate`.
4. Si aporta respuesta guiada, añade el generador en `runbook.rs`.
5. Pruebas: que **no** se dispara con datos normales, y que **sí** se dispara en el umbral.
6. Actualiza `docs/DETECCION_AMENAZAS.md`, `docs/CAPABILITIES.md`, `CHANGELOG.md` y **el número de reglas allí donde aparezca**.
7. Ejecuta `python3 scripts/guard_claims.py`: te dirá qué documento olvidaste.

Un endpoint nuevo

1. Handler en `api.rs`; añádelo al router **dentro del bloque** `protected` salvo que deba ser público.
2. Si necesita datos, el SQL va en `storage.rs`, no en el handler.
3. Prueba de integración en `tests/control_plane.rs`.
4. Documenta en `docs/API.md` y en 09.

Un dato nuevo en la telemetría

1. Campo **opcional** (`Option<T>` con `#[serde(default, skip_serializing_if)]`) en `models.rs` o `security.rs`. Si tiene que ser obligatorio, hay que subir `PROTOCOL_VERSION` y mantener la versión anterior en `SUPPORTED_PROTOCOL_VERSIONS`.

2. Recolección en el módulo correspondiente del agente.
3. **Si la recolección puede fallar, emite `CollectionGap`.** No devuelvas cero.
4. Persistencia en `storage.rs`, con migración si hace falta columna nueva. Toda columna nueva debe ser nullable o tener valor por omisión.
5. Límite en `validate_security` si es una lista.

Una vista de la consola

1. Marcado del cascarón en `console/index.html`.
2. Función `render*` en `console/app.js`, **construyendo nodos con `document.createElement`**, nunca con cadenas.
3. Estilos en `console/styles.css`.
4. Ejecuta `python3 scripts/guard_console.py`.

Cómo escribir pruebas aquí

Convención de nombres: describen el comportamiento, no la implementación.

```
✔ a_finding_stays_open_when_its_surface_could_not_be_inspected
✔ unparseable_addresses_fail_towards_the_worse_case
✘ test_classify
```

Dónde va cada tipo:

Tipo	Ubicación
Unitaria	Módulo `#[cfg(test)]` en el mismo archivo
Fixture de dominio compartida	`detect::fixtures`
Integración del API	`tests/control_plane.rs`
Extremo a extremo real	`scripts/smoke.sh`

Base de datos en pruebas: siempre `sqlite::memory:`. Cada prueba arranca limpia.

Parsers: no invoques la herramienta. Pega su salida real como literal y prueba la función pura. Así la prueba corre en los tres sistemas operativos.

Partes que requieren especial cuidado

Zona	Por qué	Qué hacer antes de tocarla
`auth.rs`, orden de comprobaciones	Mover el token delante del perímetro rompe el ADR 0003	Leer el ADR
`BindScope::classify`	Fallar hacia `Public` es deliberado	Leer la prueba `unparseable_addresses_fail_towards_the_worse_case`
`self_healing_categories`	Ampliarla sin comprobar brechas rompe REQ-SEC-003	Leer REQ-SEC-003
`upsert_incident`	Tres reglas de negocio en tres líneas: reabrir, sumar ocurrencias, conservar la severidad máxima	Leer 06 §9

Zona	Por qué	Qué hacer antes de tocarla
`probe.rs`	Añadir un programa amplía la superficie de todo el producto	Justificarlo por escrito en el PR
`runbook.rs`	Un comando destructivo hace fallar el build	Leer la prueba `no_destructive_commands`
`headers.rs`	Relajar la CSP obliga a reescribir la consola	Leer `guard_console.py`
`posture.rs`, el techo	Sin él, una emergencia se diluye en la media	Leer 06 §4
Formato de las huellas	Cambiarlo desconecta los incidentes viejos de los nuevos	Ver el `debug_assert_eq!` de `watchdog.rs`
Migraciones	No hay reversión	Toda columna nueva, nullable o con valor por omisión
`PROTOCOL_VERSION`	Solo sube si el contrato deja de ser compatible	Añadir un campo opcional **no** cuenta

Convenciones que hay que respetar

Idioma. Es la convención que más sorprende al llegar, y está en `AGENTS.md`:

- **Código y comentarios: en inglés.**
- **Interfaz, documentación y todo mensaje que lea un operador: en español.**

Un `Evidence::fact("network.listener", …)` lleva su texto en español porque lo lee una persona; el comentario que explica por qué existe esa línea va en inglés.

Comentarios. El proyecto tiene un estilo muy definido: los comentarios explican **por qué**, no qué. Comparación real del código:

```
// One spike is not saturation. Reporting it as such is how a console
// teaches its operators to ignore it.
if streak < required { return; }
```

No escribas `// comprueba si la racha es menor que la requerida`.

Formato y análisis.

```
cargo fmt --all
```

Ancho 100, 4 espacios, saltos Unix, imports ordenados. Clippy sin advertencias, en las tres plataformas.

Prohibido por el compilador: `unsafe`, `todo!()`, `unimplemented!()`, `dbg!()`.

Commits. Convencionales, en español, con cuerpo que explica el porqué. Ejemplos reales del repositorio:

```
fix: una superficie no inspeccionada ya no cierra un hallazgo
chore: eliminar el módulo rca.rs, huérfano desde la reescritura del motor
```

Cada cambio actualiza pruebas, capacidades, modelo de amenazas y changelog. Está en `AGENTS.md` y no es opcional.

Antes de abrir un PR

```
cargo fmt --all -- --check
```

```
cargo clippy --workspace --all-targets -- -D warnings
```

```
cargo test --workspace
```

```
bash scripts/smoke.sh
```

```
python3 scripts/guard_console.py && python3 scripts/guard_claims.py && python3
scripts/guard_encoding.py
```

Si los seis pasan, CI casi seguro también. Y recuerda: la puerta final de CI falla si **cualquier** control queda sin ejecutarse, no solo si falla.

Primeras tareas apropiadas

En orden creciente de dificultad. Todas salen de hallazgos reales de 15 · Riesgos y deuda técnica.

Para empezar el mismo día

Tarea	Qué aprendes	Hallazgo
Renombrar el paso de CI «Medir cobertura del núcleo de detección» a lo que realmente mide	Cómo está montado CI	R-17
Quitar de `Cargo.toml` las features `cors`, `request-id` y `set-header` de `tower-http`, y comprobar que sigue compilando	Gestión de dependencias y features	R-16
Decidir qué hacer con el directorio vacío `config/`: borrarlo, o poner una política de ejemplo con su `README.md`	Cómo se usa `--policy-file`	R-15

Primera semana

Tarea	Qué aprendes	Hallazgo
Programar `scripts/backup.sh` con un temporizador de systemd, llevarlo fuera del anfitrión y **probar la restauración**	Modelo de datos y operación	R-11
Añadir un puerto al `PORT_CATALOG` con su clase de riesgo, y su prueba	El motor de exposición de punta a punta	—
Añadir la prueba que falta: crear una base con `0001`, poblarla, aplicar `0002` y comprobar que sigue legible	Migraciones y compatibilidad	12, n.º 4
Añadir a `Database` una consulta de un solo activo y usarla en `asset_detail` en vez de listar toda la flota	Capa de almacenamiento	R-07

Primer mes

Tarea	Qué aprendes	Hallazgo
Extender el resumen en SQL a la vista de amenazas, que aún materializa filas	El corazón del producto y su compromiso de honestidad	R-07

Tarea	Qué aprendes	Hallazgo
Escribir pruebas de robustez para `parse_ss`, `parse_netstat_windows` y `parse_sshd_log` con entrada arbitraria	La zona de mayor incertidumbre técnica	R-10
Envolver el ciclo de ingesta en una transacción y probar la interrupción	Transacciones y consistencia	R-02
Añadir la opción `--agent-id` al sensor, con su validación	Identidad de activos	R-03

Lo que **no** conviene como primera tarea

- Tocar `auth.rs` o `defense.rs`: hay decisiones de diseño registradas detrás de cada línea.
- Añadir una regla de detección: primero hay que entender la política, las huellas, la auto-resolución y la honestidad de cobertura.
- Reescribir `console/app.js`: no tiene pruebas, así que una regresión pasaría inadvertida (R-09).

Itinerario resumido

Momento	Objetivo	Señal de que lo lograste
Día 1	Entender el problema y los límites	Puedes explicar por qué la misma base de datos es rutina o emergencia según dónde escuche
Día 2	Entorno funcionando	253 pruebas en verde y tu máquina enrolada en tu propia consola
Día 3	Mapa mental del repositorio	Sabes decir si algo va en `core`, en `agent` o en `server` sin buscarlo
Día 4	Un flujo de punta a punta	Puedes trazar un puerto abierto desde `ss` hasta la pantalla
Semana 1	Primera contribución	Un PR pequeño con CI en verde
Semana 2–3	Dominio de un área	Añades una regla o un endpoint sin guía
Mes 1	Autonomía	Trabajas en hallazgos de 15 sin acompañamiento

Dónde preguntar

Necesitas	Mira aquí
Qué existe y qué no	`docs/CAPABILITIES.md`
Por qué se decidió algo	`docs/adr/`
Qué debe cumplir un cambio de seguridad	`docs/requirements/`
Cómo funciona una función concreta	05 y 06
Cómo se resuelve un error	14
Cómo contribuir	`CONTRIBUTING.md`
Reportar una vulnerabilidad	`SECURITY.md`

Anterior: 17 · Resumen ejecutivo · **Siguiente:** 19 · Matriz de trazabilidad · **Índice:** README